Smile Craft Mobile

Backend Implementation Guide — Push Notifications

For: Smiles Craft backend team **Stack:** Flask + SQLAlchemy + Celery + Expo Push

Status: Required before mobile push can be enabled **Estimated effort:** ~1 working day

Contents

1. What the mobile app already does

When a user logs into the mobile app, the device generates a unique push token (an Expo Push Token, format `ExponentPushToken[xxxxxxxxxxxxxx]`). The mobile app needs to send this token to the backend so the server knows where to deliver notifications when something happens — a new appointment is booked, a patient confirms, an appointment is cancelled, and so on.

The mobile app will call `POST /register-device-token` right after a successful login, and `POST /unregister-device-token` on logout. The backend stores these tokens per user and uses them whenever it currently calls `socketio.emit(...)`.

2. Why Expo Push (and not raw FCM / APNs)

The mobile app is built on Expo. Expo provides a single push service that handles both Android (FCM under the hood) and iOS (APNs under the hood) through one HTTPS endpoint.

Approach	What the backend has to manage
Expo Push Recommended	One env var (`EXPO_ACCESS_TOKEN`), one HTTPS POST to `https://exp.host/--/api/v2/push/send`
Raw FCM	Service-account JSON, refresh tokens, Firebase project, separate certificate management
Raw APNs	`.p8` key file, Team ID, Key ID, JWT signing every hour

Expo Push is one HTTP call. No certificates to rotate, no Firebase console to maintain, no Apple Developer key renewals every year. If the clinic ever moves off Expo, the same `DeviceToken` table works — the backend simply swaps the Celery task body to call FCM / APNs directly. Nothing else changes.

3. Database schema

Add one new table. Migration is additive — no existing data is touched.

```
# models.py

class DeviceToken(db.Model):
    __tablename__ = 'device_tokens'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'),
                        nullable=False, index=True)
    token = db.Column(db.String(255), nullable=False, unique=True)
    platform = db.Column(db.String(16), nullable=False)   # 'ios' | 'android'
    device_id = db.Column(db.String(128), nullable=True)  # for de-duping per device
    created_at = db.Column(db.DateTime, default=datetime.utcnow,
                           nullable=False)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow,
                           onupdate=datetime.utcnow, nullable=False)

    user = db.relationship('User',
                           backref=db.backref('device_tokens', lazy=True))
```

Run the Flask-Migrate migration:

```
flask db migrate -m "add device_tokens table"
flask db upgrade
```

4. New endpoints

4.1 `POST /register-device-token`

Called by the mobile app immediately after login. If the same token already exists, update `user_id` and `updated_at` (the same physical device might be shared across two staff accounts). If `device_id` is provided and another row already has it, replace the token — this handles the case where a phone re-installs the app and Expo issues a new token for the same device.

```
# views.py

@main.route('/register-device-token', methods=['POST'])
@token_required
def register_device_token(current_user):
    data = request.json or {}
    token = (data.get('token') or '').strip()
    platform = (data.get('platform') or '').strip().lower()
    device_id = (data.get('deviceId') or '').strip() or None

    if not token or platform not in ('ios', 'android'):
        return jsonify({"status": "error",
                        "message": "token and platform ('ios'|'android') are require

    user_id = current_user["user_id"]

    # 1. If this device already had a token, replace it.
    if device_id:
        DeviceToken.query.filter_by(device_id=device_id).delete()

    # 2. If this token already exists (maybe under another user), reassign.
    existing = DeviceToken.query.filter_by(token=token).first()
    if existing:
        existing.user_id = user_id
        existing.platform = platform
        existing.device_id = device_id
        existing.updated_at = datetime.utcnow()
    else:
        db.session.add(DeviceToken(
            user_id=user_id,
            token=token,
            platform=platform,
            device_id=device_id,
        ))

    db.session.commit()
    return jsonify({"status": "success"})
```

4.2 `POST /unregister-device-token`

Called by the mobile app on logout, so the server stops pushing to a phone that's no longer logged in.

```
@main.route('/unregister-device-token', methods=['POST'])
@token_required
def unregister_device_token(current_user):
    data = request.json or {}
    token = (data.get('token') or '').strip()
    if not token:
        return jsonify({"status": "error", "message": "token required"}), 400

    DeviceToken.query.filter_by(token=token,
                                user_id=current_user["user_id"]).delete()
    db.session.commit()
    return jsonify({"status": "success"})
```

4.3 Mobile payload reference

This is what the mobile app will POST. No changes are needed on the backend side beyond reading these fields:

```
POST /register-device-token
Authorization: Bearer <jwt>

{
  "token": "ExponentPushToken[xxxxxxxxxxxxxxxxxxxxxx]",
  "platform": "android",
  "deviceId": "stable-device-uuid-from-expo"
}
```

5. Helper — resolve which tokens to push to

The existing socket emits target **rooms**, not user IDs:

- `clinic_<clinic_id>` — every staff member of a clinic
- `doctor_<user_id>` — one specific doctor

Add a single helper that mirrors that and returns the matching push tokens.

```
# push_helpers.py

def tokens_for_clinic(clinic_id):
    """Every active user in this clinic, across all their devices."""
    rows = (
        db.session.query(DeviceToken.token, DeviceToken.platform)
        .join(User, User.id == DeviceToken.user_id)
        .filter(User.clinic_id == clinic_id, User.status == 'active')
        .all()
    )
    return [{"token": t, "platform": p} for t, p in rows]


def tokens_for_user(user_id):
    """All devices logged in as this specific user."""
    rows = (
        db.session.query(DeviceToken.token, DeviceToken.platform)
        .filter(DeviceToken.user_id == user_id)
        .all()
    )
    return [{"token": t, "platform": p} for t, p in rows]
```

6. Celery task — `send_push_notification`

This is the single function every socket emit will call alongside its existing `socketio.emit(...)`. It runs asynchronously so HTTP requests never block the user-facing response.

```python
# tasks.py

import os
import requests
from celery import shared_task

EXPO_PUSH_URL = "https://exp.host/--/api/v2/push/send"


@shared_task(bind=True, max_retries=3, default_retry_delay=10)
def send_push_notification(self, tokens, title, body, data=None):
    """
    tokens:  list of {"token": "...", "platform": "ios"|"android"}
    title:   short heading shown in the notification banner
    body:    longer text shown under the title
    data:    optional dict — delivered to the mobile app for deep-linking
             (e.g. {"type": "newAppointment", "bookingId": 123})
    """
    if not tokens:
        return {"sent": 0}

    # Expo accepts up to 100 messages per HTTP call.
    chunks = [tokens[i:i + 100] for i in range(0, len(tokens), 100)]
    sent = 0
    errors = []

    headers = {
        "Accept": "application/json",
        "Accept-encoding": "gzip, deflate",
        "Content-Type": "application/json",
    }
    access_token = os.environ.get("EXPO_ACCESS_TOKEN")
    if access_token:
        headers["Authorization"] = f"Bearer {access_token}"

    for chunk in chunks:
        messages = [{
            "to": t["token"],
            "title": title,
            "body": body,
            "sound": "default",
            "priority": "high",
            "data": data or {},
        } for t in chunk]

        try:
            resp = requests.post(EXPO_PUSH_URL, json=messages,
                                 headers=headers, timeout=10)
            resp.raise_for_status()
            payload = resp.json()
            # Expo returns one ticket per message. Tickets with
```

```
            # status="error" and details.error == "DeviceNotRegistered"
            # mean the token is dead (user uninstalled the app) — drop it
            # from the DB.
            for ticket, msg in zip(payload.get("data", []), chunk):
                if ticket.get("status") == "ok":
                    sent += 1
                elif ticket.get("details", {}).get("error") == "DeviceNotRegistered":
                    DeviceToken.query.filter_by(token=msg["token"]).delete()
                else:
                    errors.append(ticket)
            db.session.commit()
        except requests.RequestException as exc:
            raise self.retry(exc=exc)

    return {"sent": sent, "errors": errors}
```

7. Wiring with existing socket emits

For every place that currently emits a socket event, add a matching `send_push_notification.delay(...)` call. The pattern is always the same:

```
socketio.emit(EVENT_NAME, payload, room=ROOM)

send_push_notification.delay(
    tokens_for_<clinic|user>(<id>),
    title="<short title>",
    body="<longer message>",
    data={"type": EVENT_NAME, ...payload identifiers...},
)
```

7.1 New appointment created — `views.py:2663`

```
socketio.emit("newAppointment", new_appointment, room=f"clinic_{clinic_id}")
socketio.emit("newAppointment", new_appointment, room=f"doctor_{user_id}")

send_push_notification.delay(
    tokens_for_clinic(clinic_id),
    title="New appointment",
    body=f"{patient_name} booked for {visit_date} at {start_time}",
    data={"type": "newAppointment", "bookingId": new_appointment["mainId"]},
)
```

7.2 Appointment updated / rescheduled — `views.py:2620`

```
send_push_notification.delay(
    tokens_for_clinic(clinic_id),
    title="Appointment updated",
    body=f"{patient_name}'s appointment moved to {visit_date} {start_time}",
    data={"type": "updateAppointment", "bookingId": updated_appointment["mainId
)
```

7.3 Appointment cancelled — `views.py:1461` , `3505`

```
send_push_notification.delay(
    tokens_for_clinic(clinic_id),
    title="Appointment cancelled",
    body=f"{patient_name} cancelled their {visit_date} appointment",
    data={"type": "cancelledAppointment", "bookingId": booking.id},
)
```

7.4 Patient confirmed via WhatsApp — `views.py:1438`

```
send_push_notification.delay(
    tokens_for_clinic(clinic_id),
    title="Appointment confirmed",
    body=f"{patient_name} confirmed for {visit_date}",
    data={"type": "confirmedAppointment", "bookingId": patient_appointment.id},
)
```

7.5 New patient registered — `views.py:2350`

```
send_push_notification.delay(
    tokens_for_clinic(clinic_id),
    title="New patient",
    body=f"{name} {family} was added to the clinic",
    data={"type": "patientAdded", "patientId": newPatient["id"]},
)
```

7.6 Remaining emit sites — same pattern

- **Patient edited** — `views.py:2380`
- **Patient deleted** — `views.py:2041`
- **Booking deleted** — `views.py:2500`
- **Pending bills updated** — `views.py:3722`
- **Bill paid / updated** — `views.py:3738`

Each follows the same shape — one `send_push_notification.delay(...)` call placed next to the existing socket emit. The `data` payload should carry the IDs the mobile app needs to deep-link the user to the right screen when they tap the notification.

8. Environment setup

Add one environment variable on the backend server (staging and production):

```
# .env
EXPO_ACCESS_TOKEN=<from expo.dev → Account Settings → Access Tokens>
```

> **Note —** the token is optional. Expo Push works without one, but Expo recommends using a token for production traffic. Without it, Expo may rate-limit or delay messages from unknown senders.

Steps to obtain:

1. Sign in to `https://expo.dev` with the project owner account.
2. Open **Account Settings** → **Access Tokens** → **Create token**.
3. Name it `smilescraft-prod-push`, copy the value, paste into the server's `.env`.
4. Restart the Flask and Celery workers so they pick up the new env var.

No mobile-side change is needed when the token is added.

9. Verification checklist

After deploying:

1. Mobile app installs cleanly, logs in, and `POST /register-device-token` succeeds with `{ "status": "success" }`.
2. Check the `device_tokens` table — one row per logged-in mobile device.

3. Book an appointment from the web app for a clinic that has a logged-in mobile user → mobile receives a "New appointment" banner within ~5 seconds.
4. Cancel that appointment from the web → mobile receives a "Cancelled" banner.
5. Tap the banner → mobile app opens and navigates to the appointment (handled mobile-side; backend just sends the `data` payload).
6. Log out of the mobile app → `POST /unregister-device-token` removes the row from `device_tokens`.
7. Send another web-side event → that phone no longer receives a push.

10. Scope summary

Item	Effort
`DeviceToken` model + migration	1 hour
Two new endpoints (`/register-device-token` , `/unregister-device-token`)	1 hour
Helper `tokens_for_clinic` / `tokens_for_user`	30 min
Celery task `send_push_notification` (Expo Push integration)	2 hours
Wire the task into ~10 existing socket emit sites	2 hours
Environment variable + deploy + smoke test	1 hour
Total	**~1 working day**

Once this lands on the staging backend, the mobile app's push notifications can be enabled via a single env flag flip in the next preview build — no additional mobile work required.